

Strings in Python

Technical Notes

Rodrigo Kang

Table of contents

1	Escape Sequences	5
1.1	Raw strings	6
2	Unicode and Text Representation	7
3	Strings as Sequences	8
3.1	Length	9
3.2	Indexing	10
3.3	Negative indexing	11
3.4	Slicing	11
3.5	Omitting slice boundaries	12
3.6	Negative indices in slices	13
3.7	Slice steps	14
4	Membership	15
5	A First View of String Immutability	16
6	String Operations	17
6.1	Concatenation	17
6.2	Repetition	18
7	String Comparisons	19
7.1	Lexicographical ordering	19
8	String Methods	21
8.1	Changing letter case	22
8.2	Removing surrounding characters	24
8.3	Prefixes and suffixes	25
8.4	Searching within strings	26
8.5	Counting occurrences	28
8.6	Replacing text	29
8.7	Splitting strings	30

8.8	Joining strings	31
8.9	Partitioning strings	33
8.10	Testing string contents	34
8.11	Method chaining	35
9	String Formatting	36
9.1	Formatted string literals	37
9.2	Format specifications	38
9.3	Alignment and width	39
9.4	The .format() method	40
10	Converting Between Strings and Other Types	41
10.1	Converting values to strings	42
10.2	Converting strings to integers	43
10.3	Converting strings to floating-point numbers	45
10.4	Parsing textual values	46
10.5	Conversion and validation	47
11	Strings, Bytes, and Text Encoding	49
11.1	Unicode strings	49
11.2	Encoding strings as bytes	50
11.3	Decoding bytes into strings	51
11.4	Encoding must be known	52
11.5	Why encoding matters in data work	53
12	Practical Text Normalization	54
12.1	Whitespace normalization	55
12.2	Empty strings	55
12.3	Normalization should be purpose-driven	57
13	Strings in Data Science and Machine Learning	57
13.1	Categorical data	57
13.2	Identifiers	58
13.3	Dates represented as strings	58
13.4	Natural-language text	59
14	Key Takeaways	60
15	Problems	60
15.1	Problem 1 — String literals	60
15.2	Problem 2 — Escape sequences	61
15.3	Problem 3 — Raw strings	61
15.4	Problem 4 — Unicode code points	61
15.5	Problem 5 — Indexing	62
15.6	Problem 6 — Indexing versus slicing	62
15.7	Problem 7 — Slicing	62
15.8	Problem 8 — Half-open intervals	63
15.9	Problem 9 — String immutability	63

15.10	Problem 10 — Immutability and methods	64
15.11	Problem 11 — Concatenation and types	64
15.12	Problem 12 — Lexicographical comparison	64
15.13	Problem 13 — Membership	64
15.14	Problem 14 — find() versus index()	65
15.15	Problem 15 — strip() is not substring removal	65
15.16	Problem 16 — Splitting on whitespace	66
15.17	Problem 17 — split() and join()	66
15.18	Problem 18 — Joining non-string values	66
15.19	Problem 19 — Method chaining	67
15.20	Problem 20 — lower() versus casefold()	67
15.21	Problem 21 — F-strings	67
15.22	Problem 22 — Formatting versus numerical precision	68
15.23	Problem 23 — Parsing numerical strings	68
15.24	Problem 24 — Why isdigit() is not a numeric parser	68
15.25	Problem 25 — Parsing a percentage	69
15.26	Problem 26 — Parsing formatted numbers	69
15.27	Problem 27 — Strings versus bytes	69
15.28	Problem 28 — Encoding and decoding	70
15.29	Problem 29 — Character length versus byte length	70
15.30	Problem 30 — Incorrect decoding	71
15.31	Problem 31 — Normalising categorical values	71
15.32	Problem 32 — Normalising whitespace	71
15.33	Problem 33 — Empty, whitespace, and missing values	72
15.34	Problem 34 — Numeric-looking identifiers	72
15.35	Problem 35 — Case-insensitive equality	72
15.36	Problem 36 — Palindrome detection	73
15.37	Problem 37 — Counting words	73
15.38	Problem 38 — Extracting a file extension	74
15.39	Problem 39 — Parsing key-value data	74
15.40	Problem 40 — First non-repeated character	75
15.41	Problem 41 — Character frequency	75
15.42	Problem 42 — Anagram detection	76
15.43	Problem 43 — Longest common prefix	76
15.44	Problem 44 — Run-length encoding	77
15.45	Problem 45 — Reverse the words, not the characters	77
15.46	Problem 46 — Cleaning a feature name	77
15.47	Problem 47 — Cleaning before modelling	78
15.48	Problem 48 — Data leakage through text preprocessing	79
15.49	Problem 49 — IDs versus numerical features	79
15.50	Problem 50 — Designing a robust parser	79

A **string** is a sequence of Unicode characters used to represent text. In Python, strings are instances of the built-in `str` class.

Strings are one of the most common data types in programming and data analysis. They are used to represent names, labels, categories, file paths, natural-language text, identifiers, dates

stored as text, and many other forms of non-numeric information.

A string can be created by enclosing text in either single quotes (`' '`) or double quotes (`" "`):

```
1 text_1 = "Machine Learning"
2 text_2 = 'Data Science'
3
4 type(text_1)
```

`str`

Both forms create objects of type `str`:

```
1 type(text_1) == type(text_2)
```

`True`

The choice between single and double quotes usually depends on convenience. For example, double quotes can be useful when the string itself contains an apostrophe:

```
1 sentence = "Python's syntax is relatively concise."
2
3 print(sentence)
```

Python's syntax is relatively concise.

Similarly, single quotes can be useful when the string contains double quotation marks:

```
1 sentence = 'The model returned "positive" as the predicted class.'
2
3 print(sentence)
```

The model returned "positive" as the predicted class.

Python also supports **triple-quoted strings**, written with either three single quotes or three double quotes. They can span multiple lines:

```
1 description = """Machine learning models
2 learn patterns from data
3 and use those patterns to make predictions."""
4
5 print(description)
```

Machine learning models
learn patterns from data
and use those patterns to make predictions.

Triple-quoted strings are also commonly used for documentation strings, or **docstrings**, inside functions, classes, and modules.

1 Escape Sequences

Sometimes a string must contain characters that would otherwise have a special meaning in a string literal. Python represents such characters using **escape sequences**, which begin with a backslash (\).

Some common escape sequences are:

Escape sequence	Meaning
\\	Backslash
\'	Single quote
\"	Double quote
\n	Newline
\t	Horizontal tab
\r	Carriage return

For example, \n inserts a line break:

```
1 text = "Machine Learning\nData Science"
2
3 print(text)
```

Machine Learning
Data Science

while \t inserts a horizontal tab:

```
1 text = "Feature\tValue"
2
3 print(text)
```

Feature Value

Escape sequences are also useful when a quotation mark must be included inside a string delimited by the same type of quotation mark:

```
1 sentence = 'Python\'s string type is called str.'
2
3 print(sentence)
```

Python's string type is called str.

However, choosing the other quotation style is often clearer:

```
1 sentence = "Python's string type is called str."
```

1.1 Raw strings

A **raw string** tells Python to treat backslashes primarily as literal characters instead of interpreting them as escape sequences. A raw string is created by placing `r` or `R` before the opening quotation mark:

```
1 path = r"C:\Users\Rodrigo\Documents"
2
3 print(path)
```

C:\Users\Rodrigo\Documents

This is particularly useful for Windows file paths and for expressions that contain many backslashes, such as regular expressions.

Consider the following ordinary string:

```
1 path = "C:\new_folder\test"
2 print(path)
```

C:
ew_folder est

Sequences such as `\n` and `\t` may be interpreted as newline and tab characters. The raw-string version avoids this interpretation:

```
1 path = r"C:\new_folder\test"
2 print(path)
```

C:\new_folder\test

Raw strings do not create a different data type. They are only an alternative syntax for constructing a str object:

```
1 type(r"Machine Learning")
```

str

2 Unicode and Text Representation

Python strings represent **Unicode text**. This allows strings to contain characters from many writing systems rather than being restricted to the English alphabet.

For example:

```
1 english = "Machine learning"
2 spanish = "Aprendizaje automático"
3 greek = "μηχανική μάθηση"
4 japanese = "□□□□"
5
6 print(english)
7 print(spanish)
8 print(greek)
9 print(japanese)
```

Machine learning
Aprendizaje automático
μηχανική μάθηση
□□□□

Characters with accents and other non-ASCII symbols can therefore be used directly:

```
1 city = "Córdoba"
2 country = "España"
3
4 print(city)
5 print(country)
```

Córdoba
España

Conceptually, Unicode assigns each character a unique numerical identifier called a **code point**.

Python's built-in `ord()` function returns the Unicode code point associated with a character:

```
1 ord("A")
```

65

```
1 ord("ñ")
```

241

The inverse operation can be performed with `chr()`:

```
1 chr(65)
```

'A'

```
1 chr(241)
```

'ñ'

This distinction between **characters** and their numerical representation becomes particularly important when text is stored in files or transferred between systems. A Unicode string must eventually be represented as bytes using an **encoding**, such as UTF-8.

We will return to the distinction between strings, bytes, and text encodings later in this chapter.

3 Strings as Sequences

A Python string is an **ordered sequence of characters**.

Consider the string:


```
1 language = "Python"
```

It can be viewed conceptually as:

Index	Character
0	P
1	y
2	t
3	h
4	o
5	n

Because strings are sequences, many operations that apply to Python sequences also apply to strings.

3.1 Length

The built-in `len()` function returns the number of characters contained in a string:

```
1 language = "Python"
2
3 len(language)
```

6

In this case, the string contains six characters.

Spaces are also characters:

```
1 text = "Data Science"
2
3 len(text)
```

12

Therefore, the length of a string is not necessarily equal to the number of visible letters or words.

3.2 Indexing

Individual characters can be accessed using square brackets ([]) and an integer index.

Python uses **zero-based indexing**, meaning that the first element has index 0:

```
1 language = "Python"
2
3 language[0]
```

'P'

```
1 language[1]
```

'y'

```
1 language[5]
```

'n'

Each indexed element is itself a string:

```
1 type(language[0])
```

str

Python does not have a separate built-in character type. A single character is simply a string of length one:

```
1 len(language[0])
```

1

Trying to access an index outside the valid range raises an `IndexError`:

```
1 language[6]
```

`IndexError: string index out of range`

```
-----
IndexError                                Traceback (most recent call last)
Cell In[682], line 1
----> 1 language[6]
IndexError: string index out of range
```

3.3 Negative indexing

Negative indices access characters starting from the end of the string.

The index -1 refers to the last character:

```
1 language[-1]
```

'n'

The index -2 refers to the second-to-last character:

```
1 language[-2]
```

'o'

For the string "Python", the indexing scheme can therefore be represented as:

Character	P	y	t	h	o	n
Positive index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

Negative indexing is particularly convenient when the length of the string is not known in advance.

3.4 Slicing

While indexing retrieves a single character, **slicing** retrieves a subsequence of characters.

The general syntax is:

```
1 string[start:stop]
```

where start is included and stop is excluded.

For example:

```
1 language = "Python"
2
3 language[0:3]
```

'Pyt'

returns the characters at positions 0, 1, and 2, but not the character at position 3.

This convention is known as a **half-open interval**:

$$[\text{start}, \text{stop})$$

and is used consistently throughout Python.

Because the upper bound is excluded, the length of a slice is often easy to determine:

$$\text{length of slice} = \text{stop} - \text{start}$$

provided that the indices fall within the relevant range and the default step is used.

For example:

```
1 language[2:5]
```

'tho'

contains

$$5 - 2 = 3$$

characters.

3.5 Omitting slice boundaries

The starting or stopping index can be omitted.

If start is omitted, Python begins at the start of the string:

```
1 language[:3]
```

'Pyt'

This is equivalent to:

```
1 language[0:3]
```

'Pyt'

If stop is omitted, Python continues until the end of the string:

```
1 language[3:]
```

'hon'

The entire string can therefore be written as:

```
1 language[:]
```

'Python'

Slices are more permissive than direct indexing. If a slice boundary extends beyond the available indices, Python simply returns the available portion rather than raising an `IndexError`:

```
1 language[:100]
```

'Python'

3.6 Negative indices in slices

Negative indices can also be used when slicing:

```
1 language[-3:]
```

'hon'

This returns the last three characters.

Similarly:

```
1 language[:-2]
```

'Pyth'

returns every character except the final two.

This pattern is useful when a transformation depends on positions relative to the end of a string.

3.7 Slice steps

Slicing supports a third argument called the **step**:

```
1 string[start:stop:step]
```

For example:

```
1 text = "abcdefghij"
2
3 text[0:10:2]
```

'acegi'

returns every second character in the specified interval.

When the starting and stopping positions are omitted:

```
1 text[::-2]
```

'acegi'

Python applies the step to the entire string.

A negative step traverses the string in reverse order:

```
1 text[::-1]
```

'jihgfedcba'

This provides a compact way to reverse a string.

More generally, the three slicing parameters correspond to:

start, stop, step.

The same slicing convention appears in several other Python objects and is particularly important when working with NumPy arrays and pandas objects.

4 Membership

The `in` and `not in` operators test whether a character or substring occurs within another string.

For example:

```
1 text = "machine learning"
2
3 "learn" in text
```

True

```
1 "deep" in text
```

False

The result is a Boolean value:

```
1 type("learn" in text)
```

bool

Membership testing is case-sensitive:

```
1 "Machine" in "machine learning"
```

False

The `not in` operator performs the complementary test:

```
1 "deep" not in text
```

True

These operations are frequently useful when filtering, validating, or cleaning textual data.

5 A First View of String Immutability

Although strings behave as sequences, they differ from mutable sequences such as lists in one important respect: **strings are immutable**.

Once a string has been created, its individual characters cannot be replaced in place.

For example:

```
1 language = "Python"
```

The following operation is invalid:

```
1 language[0] = "J"
```

TypeError: 'str' object does not support item assignment

TypeError Traceback (most recent call last)

Cell In[703], line 1

----> 1 language[0] = "J"

TypeError: 'str' object does not support item assignment

Python raises a `TypeError` because a `str` object does not support item assignment.

This does not mean that a variable containing a string can never change. The variable can instead be assigned to a **new string**:

```
1 language = "Python"
2 language = "J" + language[1:]
3
4 language
```

'Jython'

The original string object is not modified. A new string is constructed and the variable `language` is then associated with that new object.

This distinction becomes important when understanding string methods: methods such as `.upper()`, `.lower()`, `.replace()`, and `.strip()` do not modify the original string. They return new strings instead.

We will examine this behaviour in more detail in the following sections.

6 String Operations

Python provides several operators for working directly with strings. Some of these operators have already appeared in previous examples, but it is useful to examine their behaviour explicitly.

6.1 Concatenation

The `+` operator concatenates two strings, producing a new string containing the characters of both operands:

```
1 first_name = "Ada"
2 last_name = "Lovelace"
3
4 full_name = first_name + " " + last_name
5
6 full_name
```

'Ada Lovelace'

The operands must be strings. Python does not implicitly convert arbitrary objects to strings during concatenation:

```
1 age = 36
2
3 "Age: " + age
```

TypeError: can only concatenate str (not "int") to str

```
-----
TypeError                                Traceback (most recent call last)
Cell In[706], line 3
      1 age = 36
----> 3 "Age: " + age
TypeError: can only concatenate str (not "int") to str
```

An explicit conversion with `str()` makes the operation valid:

```
1 age = 36
2
3 "Age: " + str(age)
```

'Age: 36'

For constructing strings that contain values of other types, however, formatted strings are generally clearer. We will examine string formatting later in this chapter.

Because strings are immutable, concatenation does not modify either operand. It creates a new string:

```
1 x = "machine"
2 y = "learning"
3
4 z = x + y
5
6 print(x)
7 print(y)
8 print(z)
```

machine
learning
machine learning

6.2 Repetition

The * operator can be used to repeat a string a specified number of times:

```
1 "Python" * 3
```

'Python Python Python '

The number of repetitions must be an integer:

1 `"_" * 20`

This operation can be useful for simple text construction, although it is less common in data-processing workflows than concatenation or string methods.

7 String Comparisons

Strings can be compared using the standard comparison operators:

```
1 ==
2 !=
3 <
4 <=
5 >
6 >=
```

Equality compares the contents of two strings:

```
1 "Python" == "Python"
```

True

```
1 "Python" == "python"
```

False

String comparisons are **case-sensitive**, so uppercase and lowercase characters are considered different.

Similarly:

```
1 "Python" != "python"
```

True

7.1 Lexicographical ordering

The ordering operators compare strings **lexicographically**.

Informally, lexicographical comparison resembles dictionary ordering, but the actual comparison is based on the Unicode code points of the characters.

For example:

```
1 "apple" < "banana"
```

True

Python compares the strings character by character. In this case, the first differing characters are "a" and "b", and their Unicode code points determine the result:

```
1 ord("a"), ord("b")
```

(97, 98)

If the initial characters are equal, Python continues until it finds a difference:

```
1 "model" < "module"
```

True

Here, the comparison proceeds through the common initial characters until the first differing pair determines the ordering.

If one string is a prefix of another, the shorter string comes first:

```
1 "data" < "database"
```

True

Because comparisons ultimately depend on Unicode code points, lexicographical ordering should not be confused with linguistic or locale-specific alphabetical ordering.

For example:

```
1 "A" < "a"
```

True

The result follows from their Unicode code points:

```
1 ord("A"), ord("a")
```

(65, 97)

This is important when sorting real-world textual data. Python's default string ordering is deterministic, but it does not necessarily reproduce the alphabetical conventions of a particular natural language.

8 String Methods

The str class provides many methods for inspecting and transforming strings.

A method is called using dot notation:

```
1 string.method()
```

or, when arguments are required:

```
1 string.method(arguments)
```

Since strings are immutable, methods that appear to transform a string actually return a **new string** rather than modifying the original object.

For example:

```
1 text = "Machine Learning"
2
3 text.upper()
```

```
'MACHINE LEARNING'
```

The original string remains unchanged:

```
1 text = "Machine Learning"
2
3 text
```

```
'Machine Learning'
```

To retain the transformed value, it must be assigned:

```
1 text = "Machine Learning"
2
3 text_upper = text.upper()
4
5 text_upper
```

```
'MACHINE LEARNING'
```

There are many string methods in Python. Rather than treating them as an isolated list to memorise, it is more useful to group them according to the operations they perform.

8.1 Changing letter case

Several methods transform the case of characters.

The `.lower()` method converts cased characters to lowercase:

```
1 text = "Machine Learning"
2
3 text.lower()
```

'machine learning'

The `.upper()` method converts them to uppercase:

```
1 text = "Machine Learning"
2
3 text.upper()
```

'MACHINE LEARNING'

The `.capitalize()` method capitalizes the first character and converts the remaining cased characters to lowercase:

```
1 text = "machine LEARNING"
2
3 text.capitalize()
```

'Machine learning'

The `.title()` method converts the string to title case:

```
1 text = "introduction to machine learning"
2
3 text.title()
```

'Introduction To Machine Learning'

The `.swapcase()` method reverses the case of cased characters:

```
1 text = "Machine Learning"
2
3 text.swapcase()
```

```
'mACHINE IEARNING'
```

Another useful method is `.casefold()`:

```
1 text = "Machine Learning"
2
3 text.casefold()
```

```
'machine learning'
```

Like `.lower()`, `.casefold()` performs case normalization, but it is designed for caseless comparison of Unicode text and is therefore more aggressive for some characters.

For example:

```
1 text = "straße"
2
3 text.lower()
```

```
'straße'
```

```
1 text = "straße"
2
3 text.casefold()
```

```
'strasse'
```

This can matter when comparing text originating from different languages.

For ordinary English-language data, `.lower()` is often sufficient. When the objective is robust case-insensitive comparison of Unicode text, `.casefold()` is generally more appropriate.

8.2 Removing surrounding characters

Real-world text frequently contains unwanted whitespace around meaningful content.

The `.strip()` method removes whitespace from both ends of a string:

```
1 text = " machine learning "  
2  
3 text.strip()
```

```
'machine learning'
```

The `.lstrip()` method removes whitespace from the left:

```
1 text = " machine learning "  
2  
3 text.lstrip()
```

```
'machine learning '
```

while `.rstrip()` removes it from the right:

```
1 text = " machine learning "  
2  
3 text.rstrip()
```

```
' machine learning'
```

These methods remove surrounding whitespace, not whitespace occurring inside the string:

```
1 text = " machine learning "  
2  
3 text.strip()
```

```
'machine learning'
```

The internal spaces remain unchanged.

The methods can also receive a set of characters to remove:


```
1 text = "---machine learning---"
2
3 text.strip("-")
```

'machine learning'

An important detail is that the argument is interpreted as a **set of characters**, not as an exact prefix or suffix.

For example:

```
1 text = "www.example.com"
2
3 text.strip("w.com")
```

'example'

This behaviour may be surprising if `.strip()` is interpreted as a substring-removal operation.

When an exact prefix or suffix must be removed, Python provides dedicated methods that express the intention more clearly.

8.3 Prefixes and suffixes

The `.startswith()` method tests whether a string begins with a particular substring:

```
1 filename = "training_data.csv"
2
3 filename.startswith("training")
```

True

Similarly, `.endswith()` tests the end of the string:

```
1 filename = "training_data.csv"
2
3 filename.endswith(".csv")
```

True

Both methods return Boolean values.

They are particularly useful for validating filenames, identifiers, URLs, or other text with known structural patterns.

Python also provides `.removeprefix()` and `.removesuffix()` for removing exact prefixes and suffixes:

```
1 filename = "training_data.csv"
2
3 filename.removeprefix("training_")
```

'data.csv'

```
1 filename = "training_data.csv"
2
3 filename.removesuffix(".csv")
```

'training_data'

If the specified prefix or suffix is not present, the original string is returned:

```
1 filename = "training_data.csv"
2
3 filename.removesuffix(".txt")
```

'training_data.csv'

Unlike `.strip()`, these methods operate on the exact substring supplied.

8.4 Searching within strings

The `.find()` method searches for a substring and returns the index of its first occurrence:

```
1 text = "machine learning with Python"
2
3 text.find("learning")
```

8

If the substring is not found, `.find()` returns -1:

```
1 text = "machine learning with Python"
2
3 text.find("statistics")
```

-1

The `.rfind()` method performs the same operation but returns the index of the last occurrence:

```
1 text = "data science and data engineering"
2
3 text.rfind("data")
```

17

Python also provides `.index()` and `.rindex()`.

For a successful search, `.index()` behaves similarly to `.find()`:

```
1 text = "machine learning with Python"
2
3 text.index("learning")
```

8

The important difference appears when the substring does not exist.

`.find()` returns -1, whereas `.index()` raises a `ValueError`:

```
1 text = "machine learning with Python"
2
3 text.index("statistics")
```

`ValueError: substring not found`

```
-----
ValueError                                Traceback (most recent call last)
Cell In[746], line 3
      1 text = "machine learning with Python"
----> 3 text.index("statistics")
ValueError: substring not found
```

The appropriate method therefore depends on how the absence of the substring should be handled.

If only the existence of a substring matters, the membership operator `in` is generally clearer:

```
1 text = "machine learning with Python"
2
3 "learning" in text
```

True

rather than:

```
1 text = "machine learning with Python"
2
3 text.find("learning") != -1
```

True

The latter is valid, but the former communicates the intention more directly.

8.5 Counting occurrences

The `.count()` method returns the number of non-overlapping occurrences of a substring:

```
1 text = "data science, data engineering, data analysis"
2
3 text.count("data")
```

3

It can also be used with individual characters:

```
1 text = "mississippi"
2
3 text.count("s")
```

4

Because matching is case-sensitive:

```
1 text = "Data data DATA"
2
3 text.count("data")
```

1

Case normalization can be applied first when case should be ignored:

```
1 text = "Data data DATA"
2
3 text.lower().count("data")
```

3

This example also illustrates **method chaining**, where the result returned by one method becomes the object on which the next method operates.

8.6 Replacing text

The `.replace()` method creates a new string in which occurrences of one substring are replaced by another:

```
1 text = "machine learning with R"
2
3 text.replace("R", "Python")
```

'machine learning with Python'

The original string remains unchanged:

```
1 text = "machine learning with R"
2
3 text.replace("R", "Python")
4
5 text
```

'machine learning with R'

By default, all occurrences are replaced:

```
1 text = "data data data"
2
3 text.replace("data", "information")
```

```
'information information information'
```

An optional third argument limits the number of replacements:

```
1 text = "data data data"
2
3 text.replace("data", "information", 1)
```

```
'information data data'
```

The method performs literal substring replacement. It does not interpret regular expressions.

8.7 Splitting strings

The `.split()` method divides a string into a list of substrings.

For example:

```
1 text = "machine learning data science"
2
3 text.split()
```

```
['machine', 'learning', 'data', 'science']
```

When no separator is specified, Python splits on runs of whitespace.

This makes `.split()` without arguments more robust than explicitly splitting on a single space when the amount of whitespace may vary:

```
1 text = "machine  learning  data"
2
3 text.split()
```

```
['machine', 'learning', 'data']
```

Compare this with:

```
1 text = "machine learning data"
2
3 text.split(" ")
```

```
['machine', '', 'learning', '', '', 'data']
```

An explicit separator can be supplied when the text has a known delimiter:

```
1 record = "Alice,34,Data Scientist"
2
3 record.split(",")
```

```
['Alice', '34', 'Data Scientist']
```

This can be useful for simple structured strings, although dedicated parsers should generally be preferred for formats such as CSV when quoting, escaping, missing values, or other structural rules must be handled correctly.

The number of splits can also be limited:

```
1 record = "Alice,34,Data Scientist"
2
3 record.split(",", maxsplit=1)
```

```
['Alice', '34,Data Scientist']
```

Python provides `.rsplit()` for the corresponding operation starting from the right when a maximum number of splits is specified.

8.8 Joining strings

The inverse operation is often performed with `.join()`.

The syntax can initially appear unusual:

```
1 separator.join(iterable)
```

For example:

```

1 words = ["machine", "learning", "model"]
2
3 " ".join(words)

```

'machine learning model'

The string on which `.join()` is called acts as the separator between the elements.

A comma-separated string can therefore be constructed as:

```

1 features = ["age", "income", "balance"]
2
3 ", ".join(features)

```

'age,income,balance'

or with a comma followed by a space:

```

1 features = ["age", "income", "balance"]
2
3 ", ".join(features)

```

'age, income, balance'

The elements being joined must themselves be strings:

```

1 values = [1, 2, 3]
2
3 ", ".join(values)

```

TypeError: sequence item 0: expected str instance, int found

TypeError Traceback (most recent call last)

Cell In[765], line 3

1 values = [1, 2, 3]

----> 3 ", ".join(values)

TypeError: sequence item 0: expected str instance, int found

They can be converted explicitly when necessary:


```

1 values = [1, 2, 3]
2
3 ", ".join(str(value) for value in values)

```

'1, 2, 3'

For repeated concatenation of many strings, `.join()` also expresses the intended operation more naturally than repeatedly applying `+`.

8.9 Partitioning strings

The `.partition()` method divides a string around the first occurrence of a separator:

```

1 record = "model=linear_regression"
2
3 record.partition("=")

```

('model', '=', 'linear_regression')

It returns a tuple containing three elements:

1. the part before the separator,
2. the separator itself,
3. the part after the separator.

This makes it useful when a string contains a single meaningful delimiter:

```

1 record = "model=linear_regression"
2
3 key, separator, value = record.partition("=")
4
5 print(key)
6 print(value)

```

model
linear_regression

The corresponding `.rpartition()` method partitions around the last occurrence.

8.10 Testing string contents

The `str` class provides several predicate methods that test properties of a string and return Boolean values.

For example, `.isalpha()` tests whether the string contains alphabetic characters:

```
1 text = "Python"
2
3 text.isalpha()
```

True

```
1 text = "Python3"
2
3 text.isalpha()
```

False

The `.isdigit()` method tests whether all characters are digits:

```
1 text = "2026"
2
3 text.isdigit()
```

True

The `.isalnum()` method tests whether all characters are alphabetic or numeric:

```
1 text = "Python3"
2
3 text.isalnum()
```

True

Other useful predicates include:

```
1 text = "machine"
2
3 text.islower()
```

True

```
1 text = "MACHINE"
2
3 text.isupper()
```

True

```
1 text = " "
2
3 text.isspace()
```

True

These methods can be useful for simple validation and preprocessing tasks.

However, their meaning is based on Unicode character properties and may be broader than their names initially suggest. For example, methods such as `.isdigit()`, `.isdecimal()`, and `.isnumeric()` are related but not identical.

For many ordinary datasets this distinction is not important, but it becomes relevant when processing multilingual or heterogeneous Unicode text.

8.11 Method chaining

Because string methods return new objects rather than modifying the original string, multiple transformations can often be combined through **method chaining**.

Consider a value containing inconsistent capitalization and surrounding whitespace:

```
1 text = " MACHINE LEARNING "
```

A sequence of transformations can be written separately:

```
1 text = " MACHINE LEARNING "
2
3 text = text.strip()
4 text = text.lower()
5
6 text
```

```
'machine learning'
```

or chained:

```
1 text = " MACHINE LEARNING "  
2  
3 text.strip().lower()
```

```
'machine learning'
```

Each method operates on the result returned by the previous operation.

A more realistic cleaning example might be:

```
1 category = " Senior Data Scientist "  
2  
3 normalized_category = category.strip().casefold().replace(" ", "_")  
4  
5 normalized_category
```

```
'senior_data_scientist'
```

The result is:

```
1 senior_data_scientist
```

This pattern is common in data preprocessing, where textual categories, identifiers, column names, or labels may need to be normalized before analysis.

Method chaining can improve readability when the sequence of transformations is short and conceptually clear. Long chains, however, can become difficult to inspect and debug, in which case intermediate variables may be preferable.

9 String Formatting

Programs frequently need to construct strings that combine fixed text with values stored in variables.

Although concatenation can be used for simple cases, Python provides dedicated formatting mechanisms that are generally clearer and more expressive.

9.1 Formatted string literals

The primary modern approach to string formatting in Python is the **formatted string literal**, usually called an **f-string**.

An f-string is created by placing the prefix `f` or `F` before the opening quotation mark. Expressions enclosed in braces `{}` are evaluated and inserted into the resulting string.

For example:

```
1 name = "Ada"
2 age = 36
3
4 message = f"{name} is {age} years old."
5
6 message
```

'Ada is 36 years old.'

The expressions inside the braces do not need to be simple variable names. They can contain arbitrary Python expressions:

```
1 x = 5
2 y = 8
3
4 message = f"The sum of {x} and {y} is {x + y}."
5
6 message
```

'The sum of 5 and 8 is 13.'

Function calls can also appear inside an f-string:

```
1 model_name = "linear regression"
2
3 message = f"Model: {model_name.title()}"
4
5 message
```

'Model: Linear Regression'

This makes f-strings particularly convenient when constructing readable output from computed values.

9.2 Format specifications

F-strings support a **format specification** after a colon (:) inside the braces.

A common use is controlling the number of decimal places displayed for floating-point values:

```
1 accuracy = 0.934782
2
3 f"Accuracy: {accuracy:.2f}"
```

'Accuracy: 0.93'

Here, .2f specifies fixed-point notation with two digits after the decimal point.

Similarly:

```
1 loss = 0.00481739
2
3 f"Loss: {loss:.4f}"
```

'Loss: 0.0048'

Percentages can be formatted directly:

```
1 accuracy = 0.934782
2
3 f"Accuracy: {accuracy:.2%}"
```

'Accuracy: 93.48%'

Large numbers can be displayed with thousands separators:

```
1 observations = 1250000
2
3 f"Number of observations: {observations:,}"
```

'Number of observations: 1,250,000'

Scientific notation can also be specified:

```

1 learning_rate = 0.0000125
2
3 f"Learning rate: {learning_rate:.2e}"

```

```
'Learning rate: 1.25e-05'
```

These formatting operations affect only the textual representation of the value. They do not change the underlying numeric object:

```

1 accuracy = 0.934782
2 formatted_accuracy = f"{accuracy:.2f}"
3
4 print(formatted_accuracy)
5 print(type(formatted_accuracy))
6 print(type(accuracy))

```

```

0.93
<class 'str'>
<class 'float'>

```

The formatted result is a string, while the original value remains a floating-point number.

9.3 Alignment and width

Formatting specifications can also control the width and alignment of text.

For example:

```

1 model = "Random Forest"
2
3 f"{model:<20}"

```

```
'Random Forest      '
```

The < symbol requests left alignment within a field of width 20.

Similarly:

```

1 model = "Random Forest"
2
3 f"{model:>20}"

```

```
'    Random Forest'
```

uses right alignment, while:

```

1 model = "Random Forest"
2
3 f"{model:^20}"

```

```
' Random Forest '
```

centres the text.

These options can be useful for constructing simple textual reports or diagnostic output:

```

1 model = "Random Forest"
2 accuracy = 0.9348
3
4 print(f"{'Model':<20}{'Accuracy':>10}")
5 print(f"{model:<20}{accuracy:>10.2%}")

```

```

Model          Accuracy
Random Forest    93.48%

```

For tabular data analysis, libraries such as pandas usually provide more appropriate display mechanisms, but understanding formatting specifications remains useful for logs, reports, debugging, and command-line output.

9.4 The `.format()` method

Before f-strings were introduced, the `.format()` method was a common way to construct formatted strings.

For example:


```
1 name = "Ada"
2 age = 36
3
4 "{} is {} years old.".format(name, age)
```

'Ada is 36 years old.'

Placeholders may also refer to their positional arguments:

```
1 "{0} is {1} years old.".format("Ada", 36)
```

'Ada is 36 years old.'

and named arguments can be used:

```
1 "{name} is {age} years old.".format(name="Ada", age=36)
```

'Ada is 36 years old.'

Formatting specifications are available as well:

```
1 accuracy = 0.934782
2
3 "Accuracy: {:.2%}".format(accuracy)
```

'Accuracy: 93.48%'

The `.format()` method remains valid and is frequently encountered in existing Python code. For newly written code, however, f-strings are generally easier to read when the required formatting is straightforward.

10 Converting Between Strings and Other Types

Textual data often represents values that conceptually belong to another data type.

For example, the string:

```
1 value = "42"
2
3 type(value)
```

str

contains characters representing an integer, but the object itself is still a string.

Python provides built-in conversion functions for moving between strings and several other types.

10.1 Converting values to strings

The `str()` constructor produces a string representation of an object:

```
1 value = 42
2
3 text = str(value)
4
5 print(text)
6 print(type(text))
```

42
<class 'str'>

Floating-point values can similarly be converted:

```
1 value = 3.14159
2
3 text = str(value)
4
5 print(text)
6 print(type(text))
```

3.14159
<class 'str'>

as can Boolean values:

```
1 value = True
2
3 text = str(value)
4
5 print(text)
6 print(type(text))
```

```
True
<class 'str'>
```

The purpose of `str()` is to obtain a readable textual representation of an object. It does not preserve the original data type.

For example:

```
1 number = 42
2 text = str(number)
3
4 number == text
```

```
False
```

The objects represent related information, but an integer and its string representation are not equal.

10.2 Converting strings to integers

A string containing a valid integer representation can be converted with `int()`:

```
1 text = "42"
2
3 number = int(text)
4
5 print(number)
6 print(type(number))
```

```
42
<class 'int'>
```

Leading and trailing whitespace is accepted:

```
1 text = " 42 "  
2  
3 int(text)
```

42

However, the string must otherwise represent a valid integer:

```
1 text = "42.5"  
2  
3 int(text)
```

ValueError: invalid literal for int() with base 10: '42.5'

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[804], line 3  
      1 text = "42.5"  
----> 3 int(text)  
ValueError: invalid literal for int() with base 10: '42.5'
```

Similarly:

```
1 text = "forty-two"  
2  
3 int(text)
```

ValueError: invalid literal for int() with base 10: 'forty-two'

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[805], line 3  
      1 text = "forty-two"  
----> 3 int(text)  
ValueError: invalid literal for int() with base 10: 'forty-two'
```

Both operations raise a ValueError.

10.3 Converting strings to floating-point numbers

The `float()` constructor converts compatible textual representations to floating-point values:

```
1 text = "42.5"
2
3 number = float(text)
4
5 print(number)
6 print(type(number))
```

```
42.5
<class 'float'>
```

Integer-looking strings can also be converted:

```
1 text = "42"
2
3 float(text)
```

```
42.0
```

Scientific notation is accepted:

```
1 text = "1.25e-4"
2
3 float(text)
```

```
0.000125
```

Again, an incompatible representation raises a `ValueError`:

```
1 text = "12 kg"
2
3 float(text)
```

```
ValueError: could not convert string to float: '12 kg'
```

```
-----
ValueError
Cell In[809], line 3
```

Traceback (most recent call last)

```
1 text = "12 kg"
----> 3 float(text)
ValueError: could not convert string to float: '12 kg'
```

This distinction is important in data processing. A value may visually contain a number while still requiring cleaning before numerical conversion is possible.

10.4 Parsing textual values

Parsing is the process of interpreting textual data according to an expected structure and converting it into a more appropriate representation.

Consider:

```
1 raw_value = " 125.40 "
```

The surrounding whitespace can be removed before conversion:

```
1 raw_value = " 125.40 "
2
3 value = float(raw_value.strip())
4
5 value
```

125.4

Similarly, a percentage represented as text requires additional interpretation:

```
1 raw_percentage = "93.5%"
2
3 percentage = float(raw_percentage.removesuffix("%")) / 100
4
5 percentage
```

0.935

A currency-like representation may require multiple cleaning operations:

```
1 raw_amount = "$1,250.50"
2
3 amount = float(
4     raw_amount
5     .removeprefix("$")
6     .replace(",", "")
7 )
8
9 amount
```

1250.5

These examples illustrate an important distinction:

string cleaning changes the textual representation, while **type conversion** changes the type used to represent the underlying value.

In real datasets, these two stages frequently occur together.

10.5 Conversion and validation

A common mistake is to assume that a string can be safely converted merely because it resembles a number.

For example:

```
1 value = "123"
2
3 value.isdigit()
```

True

may appear to provide a convenient check before conversion.

However, numeric representations can take many valid forms:

```
1 "-123".isdigit()
```

False

```
1 "12.5".isdigit()
```

False

Both return False, even though the strings can be converted to numeric values:

```
1 int("-123")
```

-123

```
1 float("12.5")
```

12.5

For this reason, character predicate methods such as `.isdigit()` should not be treated as general-purpose numeric parsers.

When conversion itself determines whether a representation is valid, exception handling is often more reliable:

```
1 raw_value = "12.5"
2
3 try:
4     value = float(raw_value)
5     print(value)
6 except ValueError:
7     print("Invalid numeric representation")
```

12.5

Exception handling will be discussed separately later, but this pattern is important when processing potentially malformed input.

11 Strings, Bytes, and Text Encoding

A Python str object represents Unicode text, but computers ultimately store and transmit information as bytes.

This creates an important distinction between:

- **text**, represented in Python by str;
- **binary data**, represented in Python by bytes.

These concepts are related but not interchangeable.

11.1 Unicode strings

Consider:

```
1 text = "Córdoba"
2
3 type(text)
```

str

The object is a Unicode string.

Each character corresponds conceptually to a Unicode code point:

```
1 text = "Córdoba"
2
3 [(character, ord(character)) for character in text]
```

```
[('C', 67),
 ('ó', 243),
 ('r', 114),
 ('d', 100),
 ('o', 111),
 ('b', 98),
 ('a', 97)]
```

These code points provide an abstract representation of characters. They do not by themselves specify the sequence of bytes used to store or transmit the text.

For that, an **encoding** is required.

11.2 Encoding strings as bytes

An encoding defines how Unicode characters are represented as sequences of bytes.

UTF-8 is the dominant general-purpose encoding and can represent the full Unicode character set.

Python strings can be converted to bytes using `.encode()`:

```
1 text = "Córdoba"
2
3 encoded = text.encode("utf-8")
4
5 encoded
```

```
b'C\x33\x33\x33rdoba'
```

The result is a bytes object:

```
1 text = "Córdoba"
2 encoded = text.encode("utf-8")
3
4 type(encoded)
```

```
bytes
```

A bytes literal is displayed with a `b` prefix:

```
1 encoded
```

```
b'C\x33\x33\x33rdoba'
```

ASCII characters retain their familiar one-byte representation in UTF-8, while other characters may require multiple bytes.

This can be seen by comparing the number of Unicode characters with the number of encoded bytes:

```
1 text = "Córdoba"
2 encoded = text.encode("utf-8")
3
4 len(text), len(encoded)
```

(7, 8)

The two lengths need not be equal.

This is an important consequence of Unicode: a **character is not necessarily equivalent to one byte**.

11.3 Decoding bytes into strings

The inverse operation is **decoding**.

A sequence of bytes can be converted into Unicode text using `.decode()`:

```
1 encoded = b'C\xc3\xb3rdoba'
2
3 text = encoded.decode("utf-8")
4
5 text
```

'Córdoba'

The resulting object is again a string:

```
1 encoded = b'C\xc3\xb3rdoba'
2 text = encoded.decode("utf-8")
3
4 type(text)
```

str

The conceptual relationship is therefore:

$$\text{Unicode string} \xrightarrow{\text{encode}} \text{bytes} \xrightarrow{\text{decode}} \text{Unicode string}.$$

Encoding converts text into bytes:

```
1 str -> bytes
```

Decoding interprets bytes as text:

```
1 bytes -> str
```

Keeping this direction clear prevents a large class of text-processing errors.

11.4 Encoding must be known

Bytes do not inherently identify the encoding used to create them.

The same byte sequence may therefore be interpreted differently under different encodings.

For example:

```
1 text = "café"
2
3 encoded = text.encode("utf-8")
4
5 encoded
```

```
b'caf\xc3\xa9'
```

Decoding those bytes using UTF-8 reproduces the original text:

```
1 text = "café"
2 encoded = text.encode("utf-8")
3
4 encoded.decode("utf-8")
```

```
'café'
```

Using an incompatible encoding may raise a `UnicodeDecodeError`:

```
1 text = "café"
2 encoded = text.encode("utf-8")
3
4 encoded.decode("ascii")
```

UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in range(128)

UnicodeDecodeError Traceback (most recent call last)

Cell In[830], line 4

```
1 text = "café"
2 encoded = text.encode("utf-8")
----> 4 encoded.decode("ascii")
```

UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in range(128)

The bytes themselves are not corrupted. The error occurs because ASCII cannot interpret the complete byte sequence according to its character mapping.

11.5 Why encoding matters in data work

Encoding becomes visible whenever text crosses a boundary between systems.

Typical examples include:

- reading CSV or text files;
- importing datasets produced by external systems;
- consuming APIs;
- processing web content;
- reading database exports;
- exchanging files across operating systems;
- handling multilingual datasets.

For example, when opening a text file explicitly:

```
1 with open("data.txt", encoding="utf-8") as file:
2     text = file.read()
```

the encoding argument tells Python how the bytes stored in the file should be interpreted as Unicode characters.

Similarly, pandas allows the encoding to be specified when reading textual datasets:

```
1 import pandas as pd
2
3 df = pd.read_csv("data.csv", encoding="utf-8")
```

The general principle is:

Text should be decoded into Unicode as it enters the application and encoded back into bytes when it leaves the application.

Most ordinary Python string manipulation should therefore take place on str objects rather than directly on encoded bytes.

12 Practical Text Normalization

Strings in real datasets often contain inconsistencies that are visually small but computationally significant.

Consider categorical values such as:

```
1 category_1 = "Data Scientist"
2 category_2 = "data scientist"
3 category_3 = " Data Scientist "
```

To a human reader, these may represent the same category. Python, however, compares their exact contents:

```
1 category_1 = "Data Scientist"
2 category_2 = "data scientist"
3 category_3 = " Data Scientist "
4
5 category_1 == category_2
```

False

```
1 category_1 == category_3
```

False

Simple normalization can make the intended equivalence explicit:

```
1 category_1 = "Data Scientist"
2 category_2 = "data scientist"
3 category_3 = " Data Scientist "
4
5 normalized_1 = category_1.strip().casefold()
```

```

6 normalized_2 = category_2.strip().casefold()
7 normalized_3 = category_3.strip().casefold()
8
9 normalized_1 == normalized_2 == normalized_3

```

True

This pattern is common when cleaning categorical features.

12.1 Whitespace normalization

Whitespace can occur in forms other than an ordinary space.

For simple text containing inconsistent runs of whitespace, a useful pattern is:

```

1 text = "machine  learning\tand\ndata science"
2
3 normalized = " ".join(text.split())
4
5 normalized

```

'machine learning and data science'

Without arguments, `.split()` separates on arbitrary runs of whitespace. Joining the resulting words with a single space produces a normalized representation.

This should not be applied indiscriminately, however. Whitespace may carry structural information in some forms of text.

12.2 Empty strings

An **empty string** contains no characters:

```

1 text = ""
2
3 len(text)

```

0

An empty string evaluates to False in a Boolean context:

```
1 text = ""
2
3 bool(text)
```

False

while a non-empty string evaluates to True:

```
1 text = "data"
2
3 bool(text)
```

True

This allows concise checks such as:

```
1 text = ""
2
3 if not text:
4     print("The string is empty.")
```

The string is empty.

However, an empty string should not automatically be interpreted as a missing value.

For example, these concepts are different:

```
1 ""
2 " "
3 None
4 NaN
```

An empty string contains zero characters. A string containing a space contains one whitespace character. None represents the absence of a Python value, while NaN is commonly used by numerical and data-analysis libraries to represent missing numerical data.

Understanding these distinctions becomes important when cleaning datasets.

12.3 Normalization should be purpose-driven

Text normalization can make values easier to compare, but transformations may also remove meaningful information.

For example:

```
1 identifier_1 = "NZ-001-A"  
2 identifier_2 = "nz-001-a"
```

Converting identifiers to lowercase may be harmless if the system defines them as case-insensitive. It may be incorrect if case forms part of the identifier.

Similarly, punctuation, accents, whitespace, and capitalization may or may not carry useful information depending on the problem.

Therefore, text preprocessing should not be treated as a fixed sequence of operations.

The appropriate transformations depend on:

- the meaning of the variable;
- how the data was generated;
- the downstream analysis;
- the assumptions of the model or algorithm.

13 Strings in Data Science and Machine Learning

String manipulation appears throughout a data science workflow even when the final model operates entirely on numerical values.

13.1 Categorical data

Categorical variables frequently arrive as strings:

```
1 job_title = "Data Scientist"  
2 country = "New Zealand"  
3 segment = "Enterprise"
```

Before modelling, categories may need to be cleaned, standardized, validated, and eventually encoded numerically.

For example:

```
1 raw_segment = " ENTERPRISE "  
2  
3 segment = raw_segment.strip().casefold()  
4  
5 segment
```

'enterprise'

Normalization should happen according to the semantic meaning of the feature rather than simply because a string method is available.

13.2 Identifiers

Many identifiers should remain strings even when they contain only digits.

For example:

```
1 customer_id = "001284"
```

Converting the identifier to an integer would remove the leading zeros:

```
1 customer_id = "001284"  
2  
3 int(customer_id)
```

1284

This changes its representation and may alter its meaning.

Telephone numbers, postcodes, account numbers, product codes, and similar values are often better represented as strings because arithmetic operations on them have no meaningful interpretation.

The appearance of digits does not imply that a variable is conceptually numerical.

13.3 Dates represented as strings

Dates are also commonly received as textual values:

```
1 date = "2026-08-16"
```

Although slicing could extract individual components:

```
1 date = "2026-08-16"
2
3 year = date[:4]
4 month = date[5:7]
5 day = date[8:10]
6
7 year, month, day
```

```
('2026', '08', '16')
```

this is appropriate only for simple exploration when the format is fully known.

For production data processing, dates should generally be parsed using dedicated date and time functionality because calendar semantics are more complex than ordinary string manipulation.

13.4 Natural-language text

Natural-language data introduces additional considerations.

Operations such as:

```
1 text = "Machine Learning is Useful"
2
3 text.lower()
```

```
'machine learning is useful'
```

or:

```
1 text = "Machine Learning is Useful"
2
3 text.split()
```

```
['Machine', 'Learning', 'is', 'Useful']
```

may be useful for simple exploration, but modern natural-language processing normally requires more sophisticated treatment.

Tokenisation, Unicode normalization, punctuation handling, subword representation, vocabulary construction, and model-specific preprocessing cannot generally be reduced to simple calls to `.lower()` and `.split()`.

The string operations covered in this chapter provide the underlying Python mechanics. They should not be confused with a complete NLP preprocessing pipeline.

14 Key Takeaways

A Python string is an immutable sequence of Unicode characters represented by the `str` type.

Strings support sequence operations such as indexing, slicing, membership testing, concatenation, and repetition. Because strings are immutable, operations that appear to modify their contents instead create new string objects.

The `str` class provides methods for common text-processing tasks, including case conversion, whitespace removal, searching, replacement, splitting, joining, and validation.

Modern Python code generally uses **f-strings** to construct formatted text. Formatting changes how a value is represented as text but does not modify its underlying value or data type.

Textual representations can be converted to numerical values using constructors such as `int()` and `float()`, provided that the strings contain valid representations. Real-world parsing often requires cleaning and validation before conversion.

Python distinguishes between Unicode text (`str`) and encoded binary data (`bytes`). Encoding converts strings to bytes, while decoding converts bytes back to strings. UTF-8 is the standard encoding for most modern text-processing workflows.

Finally, textual preprocessing should be guided by the meaning of the data. Lowercasing, stripping whitespace, removing punctuation, or converting apparent numbers should not be applied mechanically. In data science and machine learning, preserving semantics is more important than merely producing a superficially uniform representation.

15 Problems

15.1 Problem 1 — String literals

Without running the code, determine the value and type of each expression:

```

1 a = "42"
2 b = '42'
3 c = ""42""
4
5 a == b == c
6 type(a)

```

Explain whether the different quotation styles affect the resulting object.

15.2 Problem 2 — Escape sequences

Predict the exact output:

```

1 text = "model\taccuracy\nlinear\t0.91"
2 print(text)

```

Explain the role of each escape sequence.

15.3 Problem 3 — Raw strings

Consider:

```

1 path_1 = "C:\new_folder\test"
2 path_2 = r"C:\new_folder\test"

```

Without running the code:

1. Explain why the two strings do not represent the same sequence of characters.
2. Predict what happens when each is printed.
3. Explain what the `r` prefix changes and what it does not change.

15.4 Problem 4 — Unicode code points

Consider:

```

1 text = "café"

```

Answer the following:

1. What does `len(text)` return?
2. What does `ord("é")` represent?
3. Is a Unicode code point the same thing as a byte?
4. Why might `len(text)` differ from `len(text.encode("utf-8"))`?

15.5 Problem 5 — Indexing

Given:

```
1 text = "machine"
```

determine the result of:

```
1 text[0]
2 text[-1]
3 text[-3]
4 text[len(text) - 1]
```

Explain the relationship between positive and negative indexing.

15.6 Problem 6 — Indexing versus slicing

Given:

```
1 text = "Python"
```

predict the result of:

```
1 text[10]
2 text[:10]
```

Why does one expression raise an exception while the other does not?

15.7 Problem 7 — Slicing

Given:

```
1 text = "machine_learning"
```

determine the result of each expression without executing the code:

```
1 text[:7]
2 text[8:]
3 text[-8:]
4 text[:-9]
5 text[::2]
6 text[::-1]
```

For each expression, explain how start, stop, and step are interpreted.

15.8 Problem 8 — Half-open intervals

Python slices use the convention:

```
1 [start:stop]
```

where start is included and stop is excluded.

Explain why this convention is useful. In particular, explain the relationship between:

```
1 text[:k]
2 text[k:]
```

and the original string.

15.9 Problem 9 — String immutability

Consider:

```
1 text = "Python"
2 text[0] = "J"
```

Answer the following:

1. What happens?
2. Why?
3. How could you construct the string "Jython" without modifying the original string in place?

15.10 Problem 10 — Immutability and methods

Predict the final values of text and result:

```
1 text = " Machine Learning "  
2 result = text.strip().lower()
```

Explain why calling `.strip()` and `.lower()` does not modify text.

15.11 Problem 11 — Concatenation and types

What happens in each case?

```
1 "accuracy = " + "0.95"
```

```
1 "accuracy = " + 0.95
```

```
1 "accuracy = " + str(0.95)
```

Explain why Python behaves differently in the three cases.

15.12 Problem 12 — Lexicographical comparison

Without running the code, determine whether each expression evaluates to True or False:

```
1 "apple" < "banana"  
2 "data" < "database"  
3 "Python" < "python"  
4 "10" < "2"
```

Explain each result in terms of lexicographical comparison and Unicode code points rather than numerical or linguistic intuition.

15.13 Problem 13 — Membership

Given:


```
1 text = "Machine Learning"
```

predict:

```
1 "Learning" in text
2 "learning" in text
3 "Machine" not in text
4 "" in text
```

Explain any result that may initially appear surprising.

15.14 Problem 14 — `find()` versus `index()`

Consider:

```
1 text = "machine learning"
```

Compare:

```
1 text.find("statistics")
2 text.index("statistics")
```

What is the behavioural difference between these methods, and when might one be preferable to the other?

15.15 Problem 15 — `strip()` is not substring removal

Without running the code, reason about the result of:

```
1 text = "www.example.com"
2
3 text.strip("w.com")
```

Why should `.strip()` not be interpreted as “remove this exact substring”?

Which type of string operation should be used when the intention is to remove an exact prefix or suffix?

15.16 Problem 16 — Splitting on whitespace

Compare the results of:

```
1 text = "machine learning\tdata\nscience"
2
3 text.split()
```

and:

```
1 text.split(" ")
```

Explain why the two operations behave differently and which is more appropriate when arbitrary runs of whitespace should be treated as separators.

15.17 Problem 17 — `split()` and `join()`

Given:

```
1 text = "machine learning model"
```

write an expression that produces:

```
1 machine_learning_model
```

Then explain the conceptual relationship between `.split()` and `.join()`.

15.18 Problem 18 — Joining non-string values

Consider:

```
1 values = [10, 20, 30]
2
3 ", ".join(values)
```

What happens and why?

Write an expression that produces:

```
1 10,20,30
```

without manually referring to each element.

15.19 Problem 19 — Method chaining

Given:

```
1 raw_category = " SENIOR DATA SCIENTIST "
```

write a single expression that produces:

```
1 senior_data_scientist
```

using only string methods.

Explain the order in which the transformations are applied.

15.20 Problem 20 — `lower()` versus `casefold()`

Explain the difference between `.lower()` and `.casefold()`.

Why might `.casefold()` be preferable when implementing case-insensitive comparisons on multilingual Unicode text?

Provide an example where the two methods do not produce the same result.

15.21 Problem 21 — F-strings

Given:

```
1 model = "Random Forest"  
2 accuracy = 0.934782  
3 observations = 1250000
```

construct a single f-string that produces output equivalent to:

```
1 Random Forest | accuracy: 93.48% | observations: 1,250,000
```

The underlying values must not be modified.

15.22 Problem 22 — Formatting versus numerical precision

Consider:

```
1 x = 0.934782
2 formatted = f"{x:.2f}"
```

Answer the following:

1. What is the type of x?
2. What is the type of formatted?
3. Has the value stored in x been rounded or modified?
4. What distinction should be made between formatting a number and changing its numerical value?

15.23 Problem 23 — Parsing numerical strings

For each string below, determine whether `int()` and `float()` can convert it directly:

```
1 "42"
2 "42.0"
3 "-42"
4 "1.25e3"
5 " 42 "
6 "42 kg"
```

Explain the result in each case.

15.24 Problem 24 — Why `isdigit()` is not a numeric parser

Consider:

```
1 values = [
2     "123",
3     "-123",
4     "12.5",
5     "1e3"
6 ]
```

Explain why `.isdigit()` is insufficient for deciding whether every value can be interpreted numerically.

What general approach would be more reliable when the goal is to parse floating-point values?

15.25 Problem 25 — Parsing a percentage

Write code that converts:

```
1 raw_value = " 87.5% "
```

into the floating-point value:

```
1 0.875
```

Your solution should correctly handle the surrounding whitespace.

15.26 Problem 26 — Parsing formatted numbers

Write code that converts:

```
1 raw_amount = "$1,250,430.75"
```

into:

```
1 1250430.75
```

Then identify which operations perform **text cleaning** and which operation performs **type conversion**.

15.27 Problem 27 — Strings versus bytes

Explain the conceptual difference between:

```
1 "café"
```

and:

```
1 b"caf\xc3\xa9"
```

Your explanation should address:

- str;
- bytes;
- Unicode code points;
- text encoding.

15.28 Problem 28 — Encoding and decoding

Given:

```
1 text = "Córdoba"
```

write code that:

1. encodes the string using UTF-8;
2. verifies that the result is a bytes object;
3. decodes the bytes back into a string;
4. verifies that the decoded value equals the original string.

15.29 Problem 29 — Character length versus byte length

Without running the code, reason about whether the following expression is necessarily True:

```
1 text = "Data Science"
2
3 len(text) == len(text.encode("utf-8"))
```

Then consider:

```
1 text = "Córdoba"
```

Would you expect the same relationship?

Explain why.

15.30 Problem 30 — Incorrect decoding

Suppose UTF-8 encoded bytes are decoded using an incompatible encoding.

What kinds of outcomes are possible?

Explain why an encoding problem is fundamentally different from an ordinary string-manipulation problem.

15.31 Problem 31 — Normalising categorical values

A dataset contains the following values for the same conceptual category:

```
1 values = [  
2     "Data Scientist",  
3     "data scientist",  
4     " DATA SCIENTIST ",  
5     "Data Scientist "  
6 ]
```

Write code that produces a consistent representation for all four values.

Then explain what assumption about the meaning of the feature makes this transformation legitimate.

15.32 Problem 32 — Normalising whitespace

Given:

```
1 text = "machine  learning\tand\ndata science"
```

produce:

```
1 machine learning and data science
```

using string methods only.

Your solution should work without knowing in advance how many spaces, tabs, or newline characters occur between words.

15.33 Problem 33 — Empty, whitespace, and missing values

Explain why the following values should not automatically be treated as equivalent:

```
1 ""
2 " "
3 None
```

How would you distinguish an empty string from a string containing only whitespace?

Why does this distinction matter when cleaning a dataset?

15.34 Problem 34 — Numeric-looking identifiers

A dataset contains:

```
1 customer_id = "001284"
```

A colleague proposes:

```
1 customer_id = int(customer_id)
```

Explain why this may be conceptually incorrect even though the conversion succeeds.

Give at least three other examples of variables that may contain only digits but should often be represented as strings.

15.35 Problem 35 — Case-insensitive equality

Implement a function:

```
1 def same_text(a, b):
2     ...
```

that returns True when two strings contain the same text ignoring:

- leading whitespace;
- trailing whitespace;
- differences in letter case.

The function should work sensibly with Unicode text.

For example:

```
1 same_text(" Straße ", "STRASSE")
```

should return True.

15.36 Problem 36 — Palindrome detection

Implement:

```
1 def is_palindrome(text):  
2     ...
```

that determines whether a string reads the same forwards and backwards.

For this problem:

- ignore letter case;
- ignore leading and trailing whitespace;
- do not use a loop.

For example:

```
1 is_palindrome("Level")
```

should return True.

15.37 Problem 37 — Counting words

Implement:

```
1 def word_count(text):  
2     ...
```

that returns the number of whitespace-separated words in a string.

The function should handle multiple spaces, tabs, and newlines correctly.

For example:

```
1 text = "machine learning\nwithPython"
```

should be counted as four words.

15.38 Problem 38 — Extracting a file extension

Given a filename such as:

```
1 filename = "model.results.final.csv"
```

extract:

```
1 csv
```

Your solution should also work when the filename contains multiple dots.

Explain why splitting at the first dot would be incorrect.

15.39 Problem 39 — Parsing key-value data

Given:

```
1 record = "model=random_forest"
```

extract the key and value into:

```
1 key = "model"
2 value = "random_forest"
```

Choose an appropriate string operation and explain why it expresses the structure of this input better than an unrestricted split.

15.40 Problem 40 — First non-repeated character

Implement:

```
1 def first_unique_character(text):  
2     ...
```

that returns the first character appearing exactly once in the string.

For example:

```
1 first_unique_character("statistics")
```

should return the first character whose total frequency in the string is one.

If no such character exists, return None.

Discuss the time complexity of your solution.

15.41 Problem 41 — Character frequency

Implement:

```
1 def character_frequencies(text):  
2     ...
```

that returns the frequency of each character in a string.

For example:

```
1 character_frequencies("data")
```

should produce a mapping equivalent to:

```
1 {  
2     "d": 1,  
3     "a": 2,  
4     "t": 1  
5 }
```

Do not use collections.Counter for the first implementation.

What is the time complexity with respect to the length of the string?

15.42 Problem 42 — Anagram detection

Implement:

```
1 def are_anagrams(a, b):  
2     ...
```

that determines whether two strings are anagrams.

For this problem:

- ignore letter case;
- ignore whitespace.

For example:

```
1 are_anagrams("Data", "A Tad")
```

should return True.

Discuss at least two possible implementation strategies and compare their time complexity.

15.43 Problem 43 — Longest common prefix

Implement:

```
1 def longest_common_prefix(strings):  
2     ...
```

that returns the longest prefix shared by every string in a list.

For example:

```
1 strings = [  
2     "machine",  
3     "matching",  
4     "material"  
5 ]
```

should return:

```
1 ma
```

Define sensible behaviour for an empty input list.

15.44 Problem 44 — Run-length encoding

Implement a simple run-length encoder:

```
1 def run_length_encode(text):  
2     ...
```

For example:

```
1 run_length_encode("aaabbcccdaa")
```

should produce a representation equivalent to:

```
1 a3b2c4d1a2
```

Discuss the time and space complexity of your implementation.

15.45 Problem 45 — Reverse the words, not the characters

Given:

```
1 text = "machine learning with Python"
```

produce:

```
1 Python with learning machine
```

Do not reverse the characters within individual words.

Your solution should also handle arbitrary runs of whitespace between words.

15.46 Problem 46 — Cleaning a feature name

Implement:

```
1 def normalize_feature_name(name):  
2     ...
```

that transforms a feature name according to the following rules:

1. remove leading and trailing whitespace;
2. convert the text to lowercase;
3. replace runs of whitespace with a single underscore;
4. remove a trailing colon if one is present.

For example:

```
1 normalize_feature_name(" Average Customer Balance: ")
```

should return:

```
1 average_customer_balance
```

15.47 Problem 47 — Cleaning before modelling

Suppose a categorical feature contains:

```
1 values = [  
2     "New Zealand",  
3     "new zealand",  
4     " NEW ZEALAND ",  
5     "New Zealand",  
6     "Australia"  
7 ]
```

Design a normalization procedure intended to make semantically equivalent categories identical.

Then answer:

1. Which transformations are clearly justified?
2. Which transformations depend on domain assumptions?
3. At what point could aggressive text normalization begin to destroy useful information?

15.48 Problem 48 — Data leakage through text preprocessing

Suppose you are building a machine-learning model and create a text preprocessing pipeline using the entire dataset before splitting it into training and test sets.

Which string-cleaning operations are inherently independent of the dataset, and which kinds of learned text preprocessing could introduce data leakage?

Explain the distinction.

15.49 Problem 49 — IDs versus numerical features

A dataframe contains the following column:

```
1 postcode
2 -----
3 0626
4 1010
5 6011
6 8011
```

A data-loading process infers the column as integers.

Discuss:

1. why this inference may be undesirable;
2. what information may be lost;
3. whether arithmetic operations on the column have meaningful interpretation;
4. how you would decide whether a variable should be treated as numeric, categorical, or textual.

15.50 Problem 50 — Designing a robust parser

You receive values such as:

```
1 values = [  
2     " 1,250.50 ",  
3     "$750.00",  
4     "N/A",  
5     "",  
6     " 92.5% ",
```

7
8

```
"-42.7"  
]
```

You need to prepare the data for a machine-learning pipeline.

Describe how you would design a robust parsing strategy.

Your answer should address:

- whitespace;
- formatting characters;
- percentages;
- missing values;
- invalid values;
- type conversion;
- error handling;
- whether all values should necessarily be interpreted using the same rule.

Do not write the complete implementation. Focus on the reasoning and design decisions.